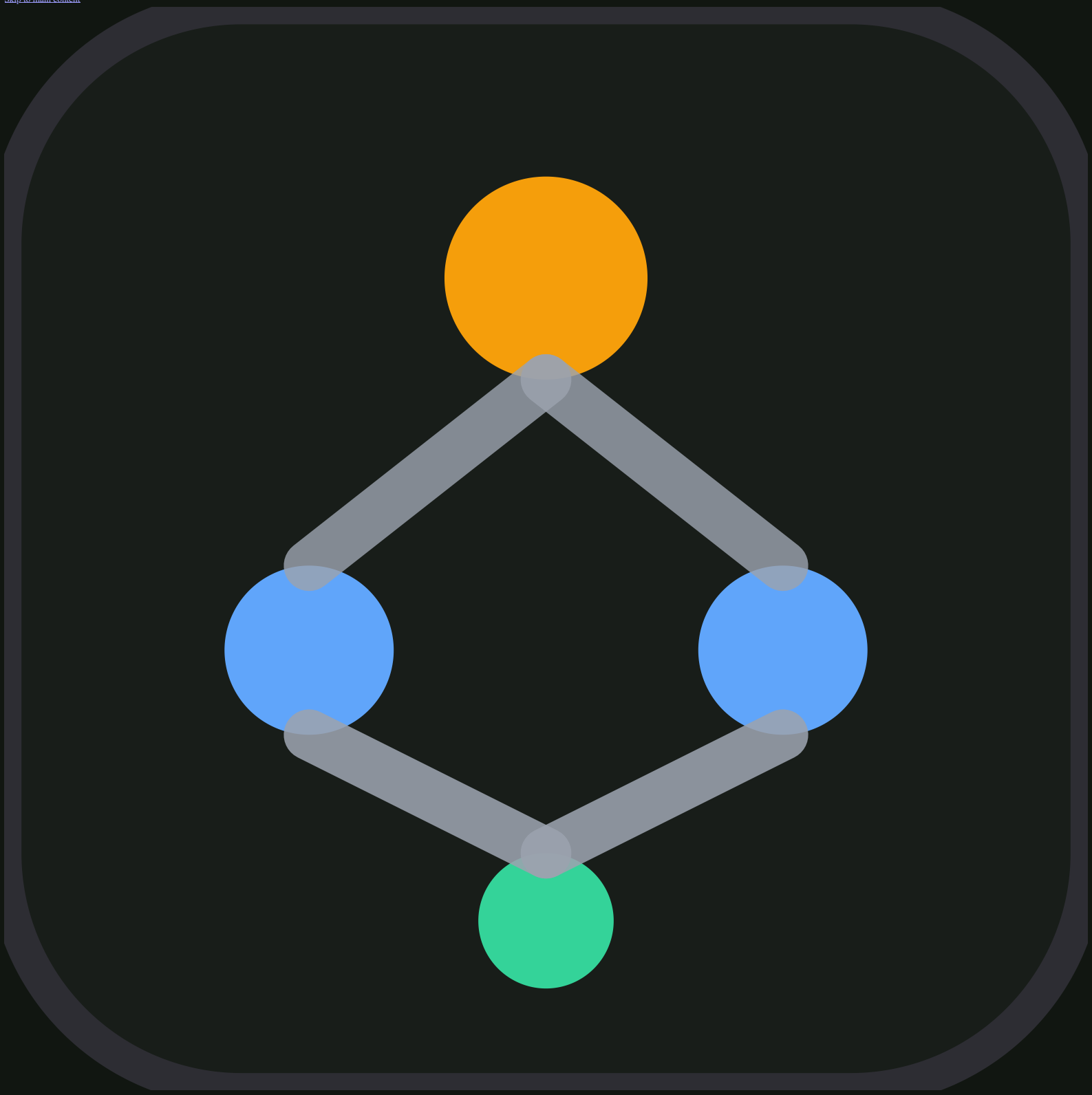

NVIDIA GPU Architectures

A concrete reference for the GPU hardware that determines what ML models can be trained and served: H100, H200, B200, GB200, and Rubin architectures, with emphasis on memory bandwidth as the primary bottleneck.

AdvancedTier 3CurrentSupporting~45 min

Prerequisites

GPU Compute ModelCuda Programming FundamentalsParallel Processing FundamentalsRunning ML Workloads on GPUS

Prereq Map

Why This Matters

Hardware dictates what is possible. The decision to train a 405B parameter model or a 70B model is not purely algorithmic. It depends on how many GPUs you have, how much memory each one holds, and how fast they can communicate. Understanding GPU architectures lets you reason about training costs, inference latency, and what techniques (quantization, tensor parallelism, offloading) are necessary for a given model at a given scale.

The key insight: for most LLM workloads, memory bandwidth is the bottleneck, not compute. Autoregressive decoding reads the entire model from memory for every single token. A GPU with more FLOPS but the same memory bandwidth does not decode faster.

Architecture Timeline

The relevant NVIDIA data center GPU generations for ML:

- **A100 (Ampere, 2020)**: 80GB HBM2e, 2 TB/s bandwidth, TF32 tensor cores
- **H100 (Hopper, 2023)**: 80GB HBM3, 3.35 TB/s bandwidth, transformer engine, FP8
- **H200 (Hopper refresh, 2024)**: 141GB HBM3e, 4.8 TB/s bandwidth, same compute as H100
- **B200 (Blackwell, 2025)**: two dies connected, FP4, second-gen transformer engine
- **GB200 NVL72 (Blackwell, 2025)**: 72 GPUs in a rack with NVLink domain

Key Specifications

Definition

HBM (High Bandwidth Memory)

HBM stacks DRAM dies vertically, connected by through-silicon vias. Each generation increases bandwidth. HBM2e (A100): 2 TB/s. HBM3 (H100): 3.35 TB/s. HBM3e (H200): 4.8 TB/s. Memory bandwidth determines inference throughput for memory-bound operations like attention and large matrix reads.

Definition

Transformer Engine

A hardware unit on Hopper and Blackwell GPUs that dynamically selects between FP8 and FP16 precision per layer during training. It monitors tensor statistics and switches precision to maintain accuracy while doubling throughput compared to FP16 on supported operations.

Definition

NVLink

A high-bandwidth interconnect between GPUs. NVLink on H100 provides 900 GB/s bidirectional bandwidth per GPU. On Blackwell NVL72, a fifth-generation NVLink connects all 72 GPUs in a single domain at 1.8 TB/s per GPU, enabling tensor parallelism across the full rack without PCIe bottlenecks.

Main Theorems

Proposition

Memory Bandwidth Bottleneck for Inference

Statement

For autoregressive LLM inference at batch size B, the time per token is approximately:

$$t_{\text{token}} \approx \max\left(\frac{2P}{B \cdot \text{BW}},\ \frac{2P}{\text{FLOPS}}\right)$$

where P is the number of parameters, BW is memory bandwidth in bytes/sec (adjusted for precision), and FLOPS is peak compute. The memory term scales as $1/B$ because each weight byte read amortizes across all B sequences in the batch. The compute term is independent of B: per-token forward work is $2P$ FLOPs regardless of how many sequences run in parallel (they share weights but not activations). For small B, the memory term dominates. The crossover batch size where compute becomes the bottleneck is $B^* = \text{FLOPS}/\text{BW}$, which is the arithmetic intensity ceiling.

Intuition

Each token generation reads the entire model from HBM (approximately $2P$ bytes for FP16). If the GPU can read at BW bytes per second, decoding one token takes at least $2P/\text{BW}$ seconds. More FLOPS do not help unless you increase batch size enough to reuse the weights you already loaded.

Proof Sketch

This follows from the roofline model. A matrix-vector product Wx with W of shape $m \times n$ costs about $2mn$ FLOPs (one FMA per weight, counted as 2 FLOPs). For FP16 inference, each weight is 2 bytes, so each byte read supports about 1 FLOP (one fused multiply-add per 2-byte weight). For FP8 this doubles to about 2 FLOP/byte, and for FP4 it is about 4 FLOP/byte. The GPU's compute-to-bandwidth ratio is typically several hundred FLOPS/byte. A single matvec is far below the roofline, so it is bandwidth bound.

Why It Matters

This explains why the H200 (same FLOPS as H100, 43% more bandwidth) is meaningfully faster for inference despite having identical compute units. It also explains why quantization to FP8 or INT4 helps inference: it reduces the bytes per parameter, directly increasing effective bandwidth.

Failure Mode

At large batch sizes, compute becomes the bottleneck. For training (large batch matmuls), the workload is compute-bound, not memory-bound. The analysis also changes for MoE models where only a fraction of parameters are active per token.

report a correction →

Architecture Details

H100 (Hopper)

- 80 GB HBM3, 3.35 TB/s bandwidth
- 989 TFLOPS FP16 tensor core, 1978 TFLOPS FP8
- Transformer engine: dynamic FP8/FP16 switching
- 900 GB/s NVLink (4th gen, 18 links)
- The baseline GPU for 2023-2024 frontier training

H200

- Same Hopper architecture and compute as H100
- 141 GB HBM3e, 4.8 TB/s bandwidth
- 76% more memory, 43% more bandwidth
- Directly improves inference latency for bandwidth-bound workloads
- Enables serving larger models without tensor parallelism

B200 (Blackwell)

- Two dies connected by a 10 TB/s chip-to-chip link
- 192 GB HBM3e, 8 TB/s bandwidth
- FP4 tensor cores: double the throughput of FP8
- Second-generation transformer engine
- Approximately 2.5x the H100 inference throughput per GPU

GB200 NVL72

- 72 Blackwell GPUs in a single rack
- NVLink connects all 72 into one domain (1.8 TB/s per GPU)
- 13.5 TB aggregate HBM across the rack
- Designed for training and serving trillion-parameter models
- Eliminates the need for InfiniBand within the rack

Rubin (Next Generation)

- Announced architecture following Blackwell
- HBM4 memory expected
- Details are preliminary as of early 2026

Roofline Regimes

The three operating regimes for LLM decode on a single GPU, parameterized by batch size B relative to the crossover $B^* = \text{FLOPS}/\text{BW}$:

REGIME	CONDITION	BOTTLENECK	PER-TOKEN TIME
Memory-bound	$B \ll B^*$	HBM reads	$2P/(B \cdot \text{BW})$
Balanced	$B \approx B^*$	Both saturated	$\approx 2P/\text{FLOPS}$
Compute-bound	$B \gg B^*$	Tensor cores	$2P/\text{FLOPS}$

For H100 FP16, $B^* \approx 295$. Single-stream chat serving runs deep in the memory-bound regime. Batched prefill and training run in the compute-bound regime.

Sparsity, Interconnect, and Alternative Accelerators

2:4 structured sparsity. Ampere and later tensor cores accept weights pruned to 2 nonzeros per 4-element block. The hardware skips the zeros, doubling effective FLOPS on sparse matmul with no bandwidth change. Practical accuracy recovery typically requires retraining with the sparsity mask fixed.

NVLink and NVSwitch. On H100, each GPU has 900 GB/s of NVLink bandwidth (18 links at 50 GB/s each, bidirectional). Inside a single node of 8 GPUs, NVSwitch provides full bisection bandwidth: any GPU can talk to any other at full NVLink speed. GB200 NVL72 extends this to 72 GPUs in one NVLink domain at 1.8 TB/s per GPU, enabling tensor parallelism across the full rack without InfiniBand hops. Collective operations (all-reduce, all-gather) scale with this fabric bandwidth, not PCIe.

TPU comparison. Google TPU v5p and v6e use systolic arrays rather than SIMT. A systolic array pushes operands through a 2D grid of MAC units on a fixed schedule, which favors static graph compilation (XLA) and dense rectangular matmuls. SIMT GPUs are more flexible for irregular kernels (attention variants, custom fused ops) at the cost of higher scheduling overhead and more complex memory hierarchies.

Common Confusions

⚠Watch Out

More FLOPS does not always mean faster inference

H200 has the same FLOPS as H100 but is faster for LLM inference because it has more memory bandwidth. For batch-size-1 autoregressive decoding, the GPU spends most of its time reading weights from HBM. Additional compute units sit idle. Only at large batch sizes does compute become the bottleneck.

⚠Watch Out

FP8 and FP4 are not just about saving memory

Lower precision halves (FP8) or quarters (FP4) the bytes read per parameter. Since inference is bandwidth-bound, this directly translates to proportionally faster decoding. The memory savings are a bonus. The primary benefit is throughput.

Summary

- Memory bandwidth, not FLOPS, determines LLM inference speed at low batch sizes
- H100: 80GB, 3.35 TB/s. H200: 141GB, 4.8 TB/s. B200: 192GB, 8 TB/s

- The transformer engine switches between precisions dynamically per layer
- NVLink bandwidth determines how efficiently you can do tensor parallelism
- GB200 NVL72 puts 72 GPUs in a single NVLink domain for trillion-parameter models

Exercises

ExerciseCore

Problem

A 70B parameter model in FP16 (2 bytes per parameter) is served on a single H100 (3.35 TB/s bandwidth). Estimate the minimum time per token for batch-size-1 autoregressive decoding.

Hint

Reveal Solution

ExerciseAdvanced

Problem

Compare the time per token for serving the same 70B FP16 model on H100 vs H200 vs B200 (assuming the model fits in memory in all cases). At what batch size does the H100 become compute-bound?

Hint

Reveal Solution

References

Canonical:

- NVIDIA H100 Tensor Core GPU Architecture Whitepaper (2022)
- NVIDIA Blackwell Architecture Technical Brief (2024)

Current:

- NVIDIA GTC 2025 Keynote, Blackwell and Rubin announcements

Next Topics

- **Flash Attention**: algorithmic optimization that reduces HBM reads
- **Fused kernels**: combining operations to minimize memory round-trips

What to do next

Test your understanding

Take the adaptive diagnostic to find your level

Where this fits

Browse curriculum

Layers 0A through 5

Find your gaps

BFS prereq checklist

Last reviewed: April 18, 2026

Canonical graph

Required before and derived from this topic

These links come from prerequisite edges in the curriculum graph. Editorial suggestions are shown here only when the target page also cites this page as a prerequisite.

Full prerequisite chainAll derived topics

Required prerequisites

4

- GPU Compute Modellayer 5 · tier 2
- Parallel Processing Fundamentalslayer 5 · tier 2
- CUDA Programming Fundamentalslayer 4 · tier 3
- Running ML Workloads on GPUslayer 4 · tier 3

Derived topics

2

- Flash Attentionlayer 5 · tier 2
- Fused Kernelslayer 5 · tier 2

Graph-backed continuations

Flash AttentionFused Kernels